



密码分析学实验报告

姓名: 邱泽辉、江李阳、王健一

学号: 202400460046、202400460042、202400460037

专业: 密码科学与技术

日期: September 16, 2026

Contents

1	实验目的	2
1.1	组员分工	2
2	实验环境	2
3	实验原理	2
3.1	字母与置换表示	2
3.2	Enigma 机的加密结构	2
3.3	接线板	3
3.4	转子组与转动规则	3
4	任务一	4
4.1	Enigma 密码机程序编写	4
4.2	测试 Enigma 密码机	4
5	任务二	4
5.1	思路分析	4
5.2	恢复 K_1 以及 K_2	5
6	任务三	6
7	实验总结	7
A	实验代码	9
A.1	Enigma 加密及性能测试程序	9
A.2	Enigma 密钥恢复程序	11
B	实验报告完整地址	17

1 实验目的

本实验围绕 Enigma 密码机的结构、加密过程和密钥空间展开。通过编程实现给定参数下的 Enigma 加密，理解接线板、转子和反射器在密码变换中的作用；在此基础上分析转子排列、转子起始位置和接线板设置所形成的密钥空间，并结合实际加密速度判断直接穷举攻击的可行性。

1.1 组员分工

- 邱泽辉（202400460046）：任务一、任务二的实现以及任务二的撰写。
- 江李阳（202400460042）：任务一、任务三的实现以及报告主体的撰写。
- 王健一（202400460037）：任务二的实现以及报告的补充、修改与整合。

2 实验环境

本实验在个人计算机上完成，使用 Python 编写 Enigma 密码机的模拟和性能测试程序。具体环境如下：

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约为 2.1 GHz；内存 16 GB。
- 软件环境：Windows 操作系统，Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。

3 实验原理

3.1 字母与置换表示

本实验将 26 个英文字母按照 $A = 0, B = 1, \dots, Z = 25$ 编码。接线板、转子和反射器都可以看作字母集合上的置换。若一个部件将输入字母对应到输出字母，则可以用一个长度为 26 的数组表示该置换。

3.2 Enigma 机的加密结构

Enigma 机主要由接线板、转子组和反射器组成。设接线板置换为 S ，三个转子在当前状态下共同形成的正向变换为 R_i ，反射器置换为 T 。其中下标 i 表示正在处理第 i 个字符时的转子状态。

对于明文字符 P_i ，其加密过程为：先经过接线板，再依次通过三个转子和反射器，随后沿相反方向通过三个转子的逆变换，最后再次经过接线板。因此有

$$C_i = S^{-1} \circ R_i^{-1} \circ T \circ R_i \circ S(P_i).$$

式中， S^{-1} 表示接线板的逆置换， R_i^{-1} 表示当前转子组的逆变换， T 表示反射器。由于接线板由成对交换组成，所以 $S^{-1} = S$ ；反射器同样由成对连接组成，因此满足 $T^{-1} = T$ 。这也是 Enigma 能够使用相同结构完成解密的原因。同时反射器满足 $T(\alpha) = \beta, \alpha \neq \beta$ ，即反射器的输入与输出必然不相等，这一特殊事件是后续对 Enigma 攻击的利用点。

3.3 接线板

接线板密钥记为 K_1 。每条接线连接两个不同字母，使两个字母互相交换；没有接线的字母映射到自身。若接线板恰好有 l 条接线，先从 26 个字母中选出参与接线的 $2l$ 个字母，再将其两两配对，因此可能的接线板数量为

$$N_{K_1}(l) = \binom{26}{2l} \frac{(2l)!}{2^l l!} = \frac{26!}{(26-2l)! 2^l l!}.$$

其中 2^l 用于消除每条接线两端交换所造成的重复计数， $l!$ 用于消除不同接线之间排列顺序造成的重复计数。按照参考程序中“不超过 6 条接线”的限制，严格的接线板密钥数量为

$$N_{K_1, \leq 6} = \sum_{l=0}^6 N_{K_1}(l) = 105\,578\,918\,576 \approx 2^{36.62}.$$

若只取接线数量最大的情况 $l = 6$ 进行数量级估计，则

$$N_{K_1}(6) = \frac{26!}{14! 2^6 6!} = 100\,391\,791\,500 \approx 2^{36.55}.$$

3.4 转子组与转动规则

转子内部的连线是预先固定的，但转子的安装顺序和起始位置会改变整体置换。在任务一中，三个转子的顺序固定为快速、中速、慢速转子 (I, II, III)，起始位置为 (8, 9, 10)。每处理一个明文字符后，快速转子向前转动一个位置；快速转子转满 26 次后，中速转子转动一个位置；中速转子转满 26 次后，慢速转子转动一个位置。

若三个转子的集合固定但顺序未知，则转子顺序有 $3!$ 种选择，每个转子的起始位置有 26 种选择，因此转子密钥数量为

$$N_{K_2} = 3! \times 26^3 = 105\,456 \approx 2^{16.69}.$$

实际使用的 Enigma 机中为了增大 K_2 的密钥空间，通常会从 t 个备选转子中选择 3 个，并按照一定初始化顺序组成实际在用的扰频器，此时该密钥的数量为：

$$N_{K_2} = P_t^3 \times 26^3$$

在任务三中，从 4 个转子中选择 3 个并安排到快速、中速、慢速三个位置，转子排列数量为

$$P_4^3 = \frac{4!}{(4-3)!} = 24.$$

因此任务三的转子密钥数量为

$$N'_{K_2} = P_4^3 \times 26^3 = 421\,824 \approx 2^{18.69}.$$

4 任务一

4.1 Enigma 密码机程序编写

按照实验指导书的配置设置 Enigma 机的加密代码，本部分在参考程序的基础上进行整理和修改，主要完成以下配置：

- 使用 5 条连线的接线板 $S = [AV BS CG MN OX]$ 设置密钥 K_1 ；
- 使用实验指导书给出的三个转子，按照快速、中速、慢速的顺序设置为 I 、 II 、 III ，并将起始位置设置为 $K_2 = (8, 9, 10)$ ；
- 设置反射器 T 。反射器是实验给定的固定部件，不作为本实验中需要恢复的密钥部分。

4.2 测试 Enigma 密码机

使用编写完成的 Enigma 机程序加密给定明文 WETTER，输出密文为 TWNVCX，运行结果如下图所示：

```
PS D:\DeskTop\CryptoAnalysis\experiment\experiment1\Experiment1_code> python .\Enigma_Encryption.py  
TWNVCX  
PS D:\DeskTop\CryptoAnalysis\experiment\experiment1\Experiment1_code> |
```

Figure 1: Enigma 机加密测试明文

由输出结果可知，程序能够按照实验指导书给定的接线板、转子、反射器和起始位置完成明文加密。

5 任务二

给定如下明密文数据流，任务 1 中的三个转子和反射器的定义不变，但转子顺序不固定，请尝试恢复密钥 K_1 和 K_2 。（假设加密过程中，中速和慢速转子不转动）

明文 WETTERVORHERSAGEHEUTE

密文 EWGZDSATLWNEDBBCJUNGW

5.1 思路分析

明文和密文长度一样，且没有出现明文和密文相同的情况，因此可以认为这就是对应的明密文，即 Crib，由于加密的时候密钥 K_1 和 K_2 是合并起来的，穷举数量太大，因此需要分割密钥空间，我们可以类比 2-DES 的攻击，其中的关键就是通过一些相同的中间量来脱去外层加密。

我们注意到这样的一种特殊的 Crib，即内部能形成环路的 Crib，因为接线板就是一个由连接线决定的单表代换，这个过程和 K_2 没有关系，enigma 机的明文输入会经过一次接线板，而输出密文前也会经过一次接线板，这之间有一个中间状态。

我们就可以利用这个中间状态单独回复 K_2 ，然后再去恢复 K_1 ，实现对于密钥空间的分割，降低密钥恢复的复杂度。

以课程上的一个 crib 为例，

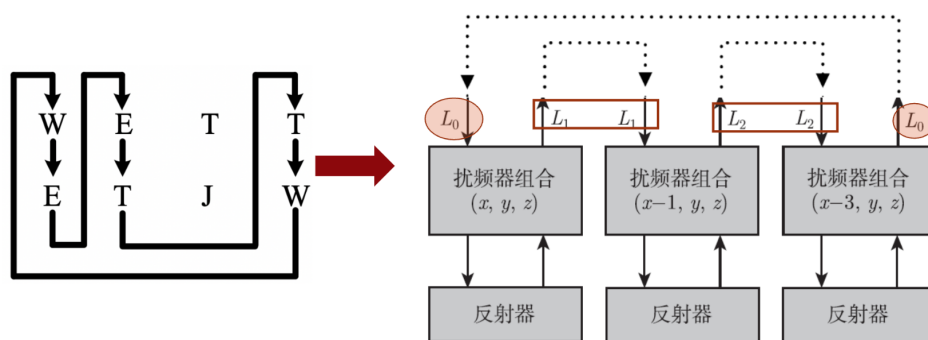


Figure 2: Crib

明文里的 W, 经过接线板, K_1 我们可以理解成一个可逆的单表带换, 即 $K_1(W) = L_0$, 然后密文的 E, $K_1(E) = L_1$, 此时我们发现明文的下一个也是 E, 它得到的中间状态也应该是 L_1 , 那这样我们就能消解掉接线板的影响

我们利用这个中间状态来建立 K_2 的方程:

它有一个**不随机现象**就是, L_0 进 L_0 出, 而 L_1 本身就是 L_0 经过扰频器和反射器得到的, 也就是说, 知道 L_0 和 K_2 , 就能计算 L_1, L_2

$$L_{i-1} \xrightarrow{K_2} L_i$$

我们简单把这个步骤记作 Enc_{K_2}

因此这中间的路径也被消解了, 我们只需要关注首尾的 L_0 , 正确的 K_2 应该能让 L_0 进 L_0 出

我们只需要遍历 $|K_2| \times |L_0|$ 次, 其中 $|L_0| = 26$, 这远远小于 $|K_1| \times |K_2|$ 的穷举

当然这个也会有错误, 但是我们多找几个环路, 然后取交集, 就能得到比较少的候选了

5.2 恢复 K_1 以及 K_2

我们前面找到的可能的 K_2 的候选集, 记为 $\{(L_0^1, K_2^1), (L_0^2, K_2^2), \dots, (L_0^n, K_2^n)\}$ 因为一个 K_2 可能对应多个 L_0 , 所以 L_0^i, K_2^i 可能有多个 i , 即多个 L_0 对应同一个 K_2

我们可以利用这些候选的 K_2 来恢复 K_1 , 同时进一步筛选 K_2

因为我们已知环路, 比如 W-E-E-T-T-W

那么就有 $K_1(W) = L_0$, $K_1(E) = L_1$, $K_1(T) = L_2$, 以此类推

如何筛选呢? 就是寻找矛盾, 再回到完整的 Crib, 寻找 K_1 中已经有的字母, 比如又找到一个 W, 它又会对应一个新的密文字母比如 J

分两种情况

- 如果 J 已经在 K_1 中出现过了, 那么我们检查 $Enc_{K_2}(K_1(W)) = K_1(J)$, 如果不等, 就排除这个候选
- 如果 J 没有出现, 那么就可以继续扩展 $K_1(J) = Enc_{K_2}(K_1(W))$

经过这轮筛选之后, 我们就能恢复出 K_1 和 K_2 了

运行 `decrypt.py` 即可得到结果

```

infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab1/report 主
oper/CommandLineTools/usr/bin/python3 /Users/infinite/Documents/crypto-analyze/lab1/decrypt.py
总的 crib 数量: 1

当前 crib 为: EWGZDSATLWNEDBBCJUNGW
当前 crib 包含的所有环路有: { (('W', 'E'), ('W', 'E'), (1, 0)), (('W', 'E', 'W'), (20, 0)), (('E', 'R', 'S', 'D', 'E'), (11, 5, 12, 4)), (('E', 'U', 'N', 'E'), (17, 18, 10)), (('T', 'G', 'T'), (19, 2)) }
【1】当前扰频器组合为: I II III
通过环路猜测的 K2 密钥个数: 524
当前扰频器组合下没有可行的 K2 和 K1

【2】当前扰频器组合为: I III II
通过环路猜测的 K2 密钥个数: 525
当前扰频器组合下没有可行的 K2 和 K1

【3】当前扰频器组合为: II I III
通过环路猜测的 K2 密钥个数: 466
通过环路及与环路相关的接线板设置无冲突, 猜测的 K2 密钥个数: 1
其中接线板总数不超过 6 条的密钥为:
(3, 14, 7) [-1, -1, 3, 2, 5, 4, 7, 6, -1, -1, -1, -1, -1, 13, -1, -1, -1, 17, 18, 19, 20, -1, 22, -1, -1, -1] 接线板条数 3
【4】当前扰频器组合为: II III I
通过环路猜测的 K2 密钥个数: 476
当前扰频器组合下没有可行的 K2 和 K1

【5】当前扰频器组合为: III I II
通过环路猜测的 K2 密钥个数: 458
当前扰频器组合下没有可行的 K2 和 K1

【6】当前扰频器组合为: III II I
通过环路猜测的 K2 密钥个数: 421
当前扰频器组合下没有可行的 K2 和 K1
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab1/report 主

```

Figure 3: result

最后 K_1 , K_2 恢复的结果是:

$$K_1 = [-1, -1, 3, 2, 5, 4, 7, 6, -1, -1, -1, -1, -1, 13, -1, -1, -1, 17, 18, 19, 20, -1, 22, -1, -1, -1]$$

$$K_2 = ((3, 14, 7), (II, I, III))$$

6 任务三

任务三要求在从 4 个转子中选择 3 个后进行加密。为便于说明, 先给出三个转子固定、但转子顺序未知时的密钥空间。

三个固定转子的排列有 $3!$ 种, 每个转子的起始位置有 26 种可能, 因此

$$N_{K_2} = 3! \times 26^3 = 105,456 \approx 2^{16.69}.$$

按照实验指导书和参考程序中的限制, 接线板最多有 6 条接线。严格计算所有允许的接线数量时,

$$N_{K_1} = \sum_{l=0}^6 \frac{26!}{(26-l)!l!} = 105,578,918,576 \approx 2^{36.62}.$$

因此, 三个转子固定但顺序未知时的完整密钥空间为

$$N = N_{K_1} N_{K_2} = 11,133,930,437,350,656 \approx 2^{53.31}.$$

若按照最大接线数 $l = 6$ 的最大项进行数量级估计, 则 $N_{K_1}(6) \approx 2^{36.55}$, 相应的总密钥空间约为 $2^{53.23}$, 同样可以近似记为 2^{53} 。

当从 4 个转子中选择 3 个并安排顺序时，转子部分的排列数量变为

$$N'_{K_2} = P_4^3 \times 26^3 = 24 \times 26^3 = 421,824 \approx 2^{18.69}.$$

接线板的可能数量不变，因此任务三的完整密钥空间为

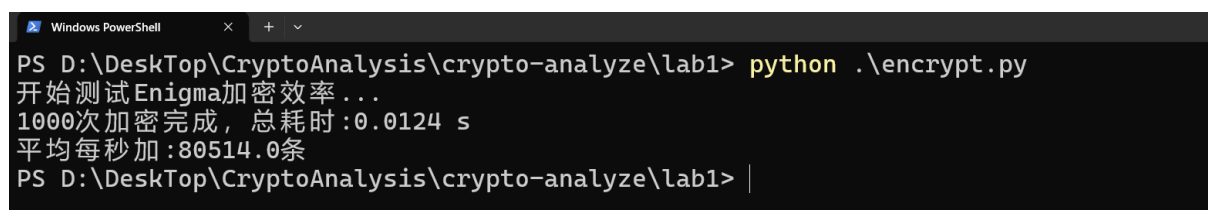
$$N' = N_{K_1} N'_{K_2} = 44,535,721,749,402,624 \approx 2^{55.31}.$$

若采用 $l = 6$ 最大项估算，则

$$N'_{l=6} = N_{K_1}(6) N'_{K_2} = 42,347,667,057,696,000 \approx 2^{55.23}.$$

因此任务三理论上需要猜测约 2^{55} 个完整密钥。

对任务一中的 Enigma 加密程序进行性能测试，采用 1000 条长度为 6 的随机明文并统计加密时间，测试结果如下：



```

PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab1> python .\encrypt.py
开始测试Enigma加密效率...
1000次加密完成，总耗时:0.0124 s
平均每秒加:80514.0条
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab1> |

```

Figure 4: 测试 Enigma 加密效率

测试结果显示每秒约可完成 80514 次加密¹。若采用严格的“不超过 6 条接线”密钥空间进行估计，则直接穷举所需时间为

$$T = \frac{N'}{80514} \approx 5.53 \times 10^{11} \text{ s} \approx 9.22 \times 10^9 \text{ min} \approx 6.40 \times 10^6 \text{ d} \approx 1.75 \times 10^4 \text{ 年}.$$

若按 $l = 6$ 最大项近似，则所需时间约为 $5.26 \times 10^{11} \text{ s}$ ，即约 1.67×10^4 年。两种计算结果均表明，在个人计算机上直接穷举任务三的完整密钥空间完全不可行。

7 实验总结

这次实验围绕 Enigma 密码机的加密实现、密钥恢复和密钥空间分析展开。通过模拟接线板、扰频器与反射器及其反向过程，我们进一步理解了 Enigma 的加密结构以及转子状态变化对加密结果的影响。在给定参数下，程序将明文 WETTER 加密为 TWNV CX，与预期结果一致，验证了该配置下加密流程的正确性。

在密钥恢复部分，我们在中速和慢速转子不转动的假设下，利用已知明密文中的特殊的 Crib 环路关系建立约束，将转子设置的搜索与接线板映射的恢复分开处理，对密钥空间进行分割。通过多个环路筛选候选，再利用完整明密文关系扩展映射、排除矛盾，最终得到转子初始位置 (3, 14, 7) 及部分接线板映射。这一过程使我们认识到，密码分析可以利用密码结构与已知信息对搜索范围进行分割，而较大的理论密钥空间并不意味着能够抵抗所有分析方法。

¹测试明文长度均为 6，随机明文在计时前生成；测试循环中复用同一台 Enigma 机器，因此测量的是连续加密吞吐量。

在密钥空间与性能分析部分，当从四个转子中选择三个并允许接线板最多使用六条接线时，总密钥空间约为 $2^{55.31}$ 。根据实验测得的连续加密吞吐量进行粗略估算，遍历完整密钥空间需要约 1.75×10^4 年，说明在本实验的实现与现有的硬件条件下，在个人计算机上直接穷举任务三的完整密钥空间完全不可行。

通过本次实验，我们加深了对于 Enigma 密码机各个部件及其作用的理解，也体会到了程序实现、结果验证与复杂度分析需要相互配合。后续可通过增加已知明密文补全尚未确定的接线板映射，并在更完整的转子转动条件下检验攻击方法的适用性。

A 实验代码

本附录列出本次实验中使用的 Enigma 加密及性能测试程序。

A.1 Enigma 加密及性能测试程序

```
1  from copy import *
2  import random
3  import time
4  class myEnigma:
5      def __init__(self, rotors, reflector) -> None:
6          """
7              初始化转子及反射器
8              Args:
9                  rotors: 转子列表, 从高速到低速
10                 reflector: 反射器
11             """
12             self.Rotor_left = [[i for i in range(26)] for j in range(len(rotors)
13             )]
14             self.Rotor_right = copy(rotors)
15             self.rotors = copy(rotors)
16             self.reflector = reflector
17
18         def setRotor(self, K2):
19             """
20                 设置转子密钥
21                 Args:
22                     K2: 转子位置密钥
23             """
24             self.count = 0
25             for i in range(len(self.Rotor_left)):
26                 self.Rotor_right[i] = self.Rotor_right[i][K2[i]:] + self.
27                 Rotor_right[i][:K2[i]]
28                 self.Rotor_left[i] = self.Rotor_left[i][K2[i]:] + self.Rotor_left
29                 [i][:K2[i]]
30
31         def resetRotor(self):
32             """
33                 重制转子位置
34             """
35             self.count = 0
36             self.Rotor_left = [[i for i in range(26)] for j in range(len(self.
37             rotors))]
38             self.Rotor_right = copy(self.rotors)
39
40         def updateRotor(self):
41             """
42                 模拟转动一次
```

```

39         """
40         self.count = self.count + 1
41         self.Rotor_left[0] = self.Rotor_left[0][25:] + self.Rotor_left
42         [0][:25]
43         self.Rotor_right[0] = self.Rotor_right[0][25:] + self.Rotor_right
44         [0][:25]
45         if self.count % 26 == 0:      #快速转子每转26次，中速转子转一次，即左移
46         一次
47             self.Rotor_left[1] = self.Rotor_left[1][25:] + self.Rotor_left
48             [1][:25]
49             self.Rotor_right[1] = self.Rotor_right[1][25:] + self.Rotor_right
50             [1][:25]
51             if self.count % (26*26) == 0: #中速转子每转26次，慢速转子转一次，即左
52             移一次
53                 self.Rotor_left[2] = self.Rotor_left[2][25:] + self.Rotor_left
54                 [2][:25]
55                 self.Rotor_right[2] = self.Rotor_right[2][25:] + self.Rotor_right
56                 [2][:25]
57
58     def encrypt(self, plaintext, K1):
59         """
60         主加密功能
61         Args:
62             plaintext: 明文字符串
63             K1: 插线板密钥
64         Return:
65             c: 密文字符串
66         """
67         plaintext = plaintext.upper() #统一大写
68         plaintext = list(plaintext)
69         ciphertext = []
70         for m in plaintext:
71             m = ord(m) - ord('A')
72             m = K1[m] #过接线板
73             for i in range(3):
74                 m = self.Rotor_right[i].index(self.Rotor_left[i][m]) #过转子
75             m = self.reflector[m] #过反射器
76             for i in range(3):
77                 m = self.Rotor_left[2 - i].index(self.Rotor_right[2 - i][m]) #
78             逆向过转子
79             m = K1[m] #过接线板
80             self.updateRotor() #每加密一个字母，更新转子
81             ciphertext.append(chr(m + ord('A')))
82         return ''.join(ciphertext)
83
84 random.seed(202698) # 设置日期为种子
85
86 def _generate_random_m(n=6):
87     # 生成随机明文供测试

```

```

79     # n表示明文长度，默认与测试明文长度相同
80     temp = []
81     for i in range(n):
82         temp.append(chr(random.randint(0,25) % 26 + ord('A')))
83     return ''.join(temp)
84
85 def test_efficiency(S,QUICK,MID,SLOW,T,K2,n=1000):
86     # 进行n=1000次加密随机明文测试效率
87     E=myEnigma([QUICK, MID, SLOW], T)
88     E.setRotor(K2)
89     M = [_generate_random_m() for _ in range(n)]
90     print("开始测试Enigma加密效率...")
91     start = time.perf_counter()
92     for m in M:
93         c = E.encrypt(m,S)
94     end = time.perf_counter()
95     print(f"{n}次加密完成，总耗时:{end-start:.4f} s")
96     print(f"平均每秒加:{n/(end-start):.1f}条")
97
98
99 if __name__=="__main__":
100     ## S = [AV BS CG MN OX]
101     S = [21,18,6,3,4,5,2,7,8,9,10,11,13,12,23,15,16,17,1,19,20,0,22,14,24,25]
102     QUICK = [7, 19, 3, 22, 11, 25, 14, 1, 16, 23, 8, 20, 5, 17, 12, 9, 24, 6,
103             15, 2, 18, 21, 4, 13, 10, 0]
104     MID = [0, 10, 4, 2, 8, 1, 18, 20, 22, 19, 13, 6, 17, 5, 9, 3, 24, 14, 12,
105           25, 21, 11, 7, 16, 15, 23]
106     SLOW = [0, 23, 5, 12, 18, 3, 21, 9, 14, 1, 17, 6, 24, 11, 20, 4, 15, 8,
107            22, 7, 19, 13, 2, 16, 10, 25]
108     T = [5, 3, 7, 1, 8, 0, 9, 2, 4, 6, 12, 14, 10, 15, 11, 13, 18, 20, 16,
109         21, 17, 19, 24, 25, 22, 23]
110     K2 = [8,9,10]
111     # E=myEnigma([QUICK, MID, SLOW], T)
112     # E.setRotor(K2)
113     # plaintext="WETTER"
114     # ciphertext=E.encrypt(plaintext, S)
115     # print(ciphertext)
116     test_efficiency(S,QUICK,MID,SLOW,T,K2)

```

Listing 1: encrypt.py

A.2 Enigma 密钥恢复程序

```

1  ...
2  给定如下明密文数据流，中的三个转子和反射器的定义不变，尝试恢复密钥K1和K2。（结果
   可能不唯一）
3  明文：  HELLOWORLD...
4  密文： ...WELLDONEEFGHIJ...

```

已知K1中接线板的接线数量最多不超过六条；
 对于K1中没有信息支撑的接线板情况，可以考虑使用-1表示未知：
 例如[0,2,1,-1,...]表示A不接线(自身到自身)，B与C相连，-1则表示字母D为自由(是否接线且与哪个字母接线未知)。

扰频器组合(固定3个转子，无需从5个中选择3个)

快速转子I = [0, 18, 24, 12, 10, 20, 8, 6, 14, 2, 11, 15, 22, 3, 25, 7, 17, 13, 5, 1, 23, 9, 16, 21, 19, 4]

即快速转子的表格中左侧一列为0, 1, 2, ..., 25, 右侧一列0, 18, 24, ..., 4; 下同

中速转子II = [0, 10, 4, 2, 8, 1, 18, 20, 22, 19, 13, 6, 17, 5, 9, 3, 24, 14, 12, 25, 21, 11, 7, 16, 15, 23]

反射器

T = [10, 20, 14, 8, 25, 15, 16, 21, 3, 18, 0, 23, 13, 12, 2, 5, 6, 19, 9, 17, 1, 7, 24, 11, 22, 4]

即a与k相连, b与u相连,

'''

```
import string
```

```
from copy import *
```

```
def crypt_once(I, II, III, T, st0, st1, st2, ch):
```

```
    """
```

```
    在指定三个转子位置下，只加密一个字符。
```

```
    不经过接线板 K1，不自动转动转子。
```

```
    Args:
```

```
        I: 快速转子
```

```
        II: 中速转子
```

```
        III: 慢速转子
```

```
        T: 反射器
```

```
        st0: 快速转子当前位置
```

```
        st1: 中速转子当前位置
```

```
        st2: 慢速转子当前位置
```

```
        ch: 输入字符，如 'A'
```

```
    Return:
```

```
        加密后的字符，如 'G'
```

```
    """
```

```
    # A -> 0, B -> 1, ...
```

```
    x = ord(ch) - ord('A')
```

```
    rotors = [I, II, III]
```

```
    positions = [st0, st1, st2]
```

```
    rotor_left = []
```

```
    rotor_right = []
```

```
    # 根据当前位置旋转三个转子
```

```
50     for rotor, pos in zip(rotors, positions):
51         pos %= 26
52         left = list(range(26))
53         left = left[pos:] + left[:pos]
54         right = rotor[pos:] + rotor[:pos]
55         rotor_left.append(left)
56         rotor_right.append(right)
57     # 过三轮
58     for i in range(3):
59         x = rotor_right[i].index(
60             rotor_left[i][x]
61         )
62     x = T[x]
63     # 逆向过三轮
64     for i in range(2, -1, -1):
65         x = rotor_left[i].index(
66             rotor_right[i][x]
67         )
68     return chr(x + ord('A'))
69 def find_crib(c,m):
70     """
71     查找crib
72     Args:
73         m: 明文字符串
74         c: 密文字符串
75     Return:
76         cribs: 所有可用cribs
77     """
78
79     # find the crib
80     cribs = []
81     maxl = len(c) - len(m) + 1
82     ccount = 0 #密文起始点
83     flag = 0
84     while(ccount < maxl):
85         temp_c = c[ccount : ccount + len(m)]
86         ccount = ccount + 1
87         for i in range(len(m)):
88             if m[i] == temp_c[i]: # 有重复的, 一定不是crib
89                 flag = 1
90                 break
91         if flag == 0:
92             cribs.append(temp_c)
93         else:
94             flag = 0
95     return cribs
96
97
98 def build_graph(c,m): #邻接表
```

```

99     """
100     查找crib
101     Args:
102         m: 明文字符串
103         c: 密文字符串
104     Return:
105         graph: 用于dfs的图, 边权为位置
106     """
107     graph = {}
108     # 遍历所有的边
109     for l, uv in enumerate(zip(c, m)):
110         u, v = uv
111         # 如果u不在字典中, 就创建一个空列表
112         if u not in graph:
113             graph[u] = []
114         # 把(v, l)这个元组添加到u的邻接列表中, 边权l是边的坐标 坐标就是索引, l=0就是c的第一个字母
115         graph[u].append((v, l))
116     for l, uv in enumerate(zip(m, c)):
117         u, v = uv
118         # 如果u不在字典中, 就创建一个空列表
119         if u not in graph:
120             graph[u] = []
121         # 把(v, l)这个元组添加到u的邻接列表中, 边权l是边的坐标
122         graph[u].append((v, l))
123     # 返回字典
124     return graph
125
126
127 def dfs(graph, pos, st, path, locs): # 从左到右依次为构建的无向图, 输入位置字符, 开始位置字符和坐标, 路径, 记录的环路构成边的坐标
128     """
129     查找crics
130     Args:
131         graph: build graph得到的图
132         pos: 当前遍历的字符
133         st: 开始位置字符和坐标
134         path: 路径
135         locs: 记录的环路构成边的坐标
136     Return:
137         ans: 所有可用crics
138     """
139
140     ans = []
141     for i, l in graph[pos]:
142         if i in path:
143             tmp = (path[path.index(i):] + [i], locs[path.index(i):] + [l])
144             if len(set(tmp[1])) >= 2 and st[0] not in tmp[0][1:-1] and tmp[1][-1] == st[1]:

```

```

145         ans+=[(tuple(tmp[0]),tuple(tmp[1]))]
146         continue
147         ans+=dfs(graph,i,st,path+[i],locs+[1])
148     return ans
149 def get_keys(d, value):
150     return [k for k,v in d.items() if v == value][0]
151
152
153 I = [7, 19, 3, 22, 11, 25, 14, 1, 16, 23, 8, 20, 5, 17, 12, 9, 24, 6, 15, 2,
154      18, 21, 4, 13, 10, 0]
155 II = [0, 10, 4, 2, 8, 1, 18, 20, 22, 19, 13, 6, 17, 5, 9, 3, 24, 14, 12, 25,
156       21, 11, 7, 16, 15, 23]
157 III = [0, 23, 5, 12, 18, 3, 21, 9, 14, 1, 17, 6, 24, 11, 20, 4, 15, 8, 22, 7,
158        19, 13, 2, 16, 10, 25]
159 T = [5, 3, 7, 1, 8, 0, 9, 2, 4, 6, 12, 14, 10, 15, 11, 13, 18, 20, 16, 21,
160      17, 19, 24, 25, 22, 23]
161
162 if __name__=="__main__":
163     m="WETTERVORHERSAGEHEUTE"
164     c="EWGZDSATLWNEDBBCJUNGW"
165     cribs=find_crib(c,m)
166     print("总的crib数量: ", len(cribs))
167     for crib in cribs:
168         print()
169         print("当前crib为: ", crib)
170         ans=set() # 保存当前crib中包含的所有环路
171         graph=build_graph(crib,m)
172         visited=[0]*len(m)
173         for i in range(len(m)):
174             if visited[i]==1:
175                 continue
176             ansi=dfs(graph,m[i],(m[i],i),[m[i]],[])
177             for find_ins in ansi:
178                 for loc in find_ins[1]: # 防止重复计算
179                     visited[loc]=1
180             ans|=set(ansi)
181         print("当前crib包含的所有环路有: ", ans)
182         order = {'I':I,'II':II,'III':III}
183         num = 1
184         for I, II, III in [(I, II, III),
185                          (I, III, II),
186                          (II, I, III),
187                          (II, III, I),
188                          (III, I, II),
189                          (III, II, I)]:
190             print(f"【{num}】当前扰频器组合为: ", get_keys(order,I), get_keys(
191 order,II), get_keys(order,III))
192             num += 1
193         guess_all_K2 = {(i, j, k) for i in range(26) for j in range(26)

```

```

189     for k in range(26)} # 从所有备选中进行筛选
190     guess={}
191     for case in ans:
192         guess_case={}
193         for i in range(26):
194             for j in range(26):
195                 for k in range(26):
196                     #这里要保存所有路径
197                     paths = []
198                     for check in string.ascii_uppercase: #枚举所有字母, 亮灯
199                         trans=check #注意这个check就是已经过了接线板的
200                         path=[trans]
201                         for z in case[1]:
202                             trans=crypt_once(I,II,III,T,(i-z)%26,j,k,
203                             trans)
204                             path.append(trans)
205                             if trans==check:
206                                 paths.append(path)
207                             if len(paths)>0:
208                                 guess_case[(i,j,k)]=paths #保存所有路径, 这个
209                                 path就是[L0,L1,L2,...]
210             guess_all_K2=set(guess_case) #每个环路会有一个guess_case,得到
211             很多ijk, 然后取交集
212             guess[case]=guess_case #guess[case]是一个字典, guess[case
213             ][(i,j,k)]=path
214             print("通过环路猜测的K2密钥个数: ",len(guess_all_K2))
215
216     #每个环都有很多路径, 得有回溯搜索
217     def merge_K1(K1, chars, path):
218         """
219         尝试把当前环的 path 合并进 K1
220         有冲突返回 None
221         """
222         new_K1 = deepcopy(K1)
223         for real, plugged in zip(chars, path):
224             # real -> plugged
225             if real in new_K1:
226                 if new_K1[real] != plugged:
227                     return None
228             else:
229                 new_K1[real] = plugged
230             # plugged -> real
231             if plugged in new_K1:
232                 if new_K1[plugged] != real: #矛盾的都排除
233                     return None
234             else:
235                 new_K1[plugged] = real
236         return new_K1

```

```

232     def search_K1(cases, guess, K2, idx=0, K1=None):
233         if K1 is None:
234             K1 = {}
235             # 所有环都处理完
236             if idx == len(cases):
237                 return [K1]
238             case = cases[idx] #这里case是形如(('T', 'G', 'T'), (19, 2)),
这样的数据结构, case[0]就是环路
239             results = []
240             # 当前环可能有多个闭环 path
241             for path in guess[case][K2]: #一个path对应可以找到一个K1的可能
性
242                 new_K1 = merge_K1(K1, case[0], path)
243                 if new_K1 is not None:
244                     results += search_K1(cases, guess, K2, idx + 1, new_K1)
245             return results
246         guess_K2_K1 = {}
247         cases = list(ans)
248         for guess_K2 in guess_all_K2:
249             K1_candidates = search_K1(cases, guess, guess_K2)
250             if K1_candidates:
251                 guess_K2_K1[guess_K2] = K1_candidates
252         if len(guess_K2_K1) == 0:
253             print("当前扰频器组合下没有可行的K2和K1\n")
254             continue
255         print("通过环路及与环路相关的接线板设置无冲突, 猜测的K2密钥个数: ",
len(guess_K2_K1))
256
257         print("其中接线板总数不超过6条的密钥为: ")
258         for guess in guess_K2_K1:
259             count=0
260             guess_K1=guess_K2_K1[guess][0]
261             K1=[-1]*26 # -1代表现有信息无法支撑获取该位置的接线板情况
262             for i in guess_K1:
263                 if guess_K1[i]!=i:
264                     count+=1
265                 K1[ord(i)-ord("A")]=ord(guess_K1[i])-ord("A")
266             if count/2>6:
267                 continue
268             print(guess, K1, "接线板条数", count//2)

```

Listing 2: decrypt.py

B 实验报告完整地址

实验完整代码、实验报告 L^AT_EX 源代码见仓库: <https://github.com/infinitepwn/crypto-analyze.git>